OncoPilot Multi-Agent DB Plan – Collection-Centric Revision

> **Goal** Extend the original MCP & multi-agent architecture so that every *database collection* is represented by a dedicated specialist agent. A central **Query-Orchestrator** decomposes end-user questions, delegates sub-tasks to these specialists, then aggregates and summarises the combined evidence.

---

1. Collections, Purpose & Specialist Agents

Database Collection	Specialist Agent	Purpose / Core Responsibility
`cancers`	**CancerAgent**	Retrieve structured data on cancers and their sub-types, associated biomarkers, incidence & prevalence metrics, staging calculators, and links to major clinical guidelines / calculators.
`genes`	**GeneAgent**	Fetch gene-centric clinical & scientific metadata: cancer associations, co-mutations, prognosis impact, linked drugs & trials, and regulatory notes about actionable variants.
`drugs`	**DrugAgent**	Access comprehensive cancer-drug information: pharmacologic class, biomarker relevance, global approvals, efficacy summaries, multi-geography trial data, and links to regulatory / clinical guidelines.
`drugs_india`	**DrugIndiaAgent**	Provide detailed data on cancer drugs available in India: CDSCO approval status, biomarkers, cancer indications, local pricing/availability, supporting evidence from Indian & global sources.

Each specialist knows *only* the schema & business logic of its home collection and owns a tight set of MCP **tools** (e.g. `cancers.search_cancer`, `genes.get_gene_variants`).

---

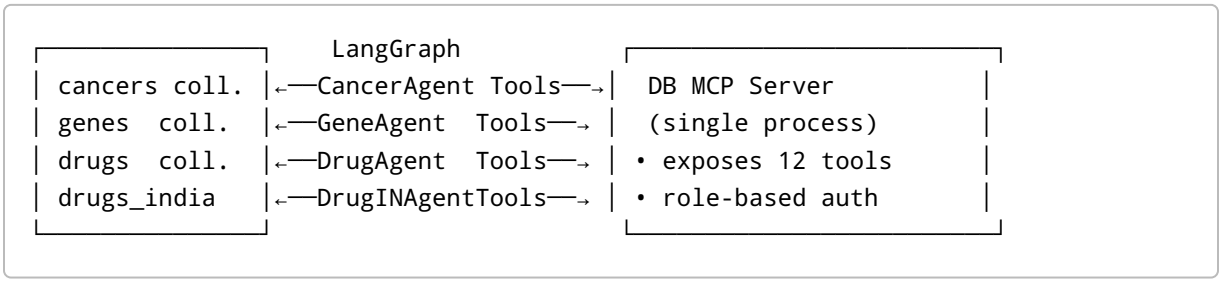
2. Query-Orchestrator Agent

- **Context window** is primed with capsule schema docs for *all* four collections so it grasps their data vocabulary.
- Implements a *ReACT-style* planner inside **LangGraph**:
- **Analyse** user query → derive atomic intents & target entity types.
- **Route** each intent to the matching Specialist Agent (parallel when possible).
- **Collect** JSON / structured payloads returned by sub-agents.

- **Synthesise** a markdown answer + sources; stream back to chat UI.
- Maintains a light memory store of recent sub-results to avoid duplicate DB hits in the same session.

---

3. MCP & Tool Layer (unchanged)



---

4. End-to-End Query Flow

```
graph TD
    UserQuery["User\nquestion"] -->|1. send| MainAgent["Query-Orchestrator"]
    MainAgent -->|2a. cancer terms| CancerAgent
    MainAgent -->|2b. gene terms| GeneAgent
    MainAgent -->|2c. drug terms| DrugAgent
    MainAgent -->|2d. Indian drug terms| DrugIndiaAgent
    CancerAgent -->|3.| Cancers[(cancers)]
    GeneAgent -->|3.| Genes[(genes)]
    DrugAgent -->|3.| Drugs[(drugs)]
    DrugIndiaAgent -->|3.| DrugsIN[(drugs_india)]
    Cancers -->|4.| CancerAgent
    Genes -->|4.| GeneAgent
    Drugs -->|4.| DrugAgent
    DrugsIN -->|4.| DrugIndiaAgent
    CancerAgent -->|5.| MainAgent
    GeneAgent -->|5.| MainAgent
    DrugAgent -->|5.| MainAgent
    DrugIndiaAgent -->|5.| MainAgent
    MainAgent -->|6. summary| Answer["Chatbot\nreply"]
```

---

5. Tool Design Examples (pseudo-Python)

```python
@mcp.tool(namespace="cancers")
def search_cancer(name: str, fields: list[str] = ["summary", "incidence"]):
    return db.cancers.find_one({"name": name}, {f: 1 for f in fields})
```

```
@mcp.tool(namespace="genes")
def get_gene_variants(hugo_symbol: str):
    return db.genes.aggregate([...])
```

---

6. Implementation Checklist (unchanged core steps)

1. **Refactor DB MCP Server** with namespaced tools.
2. **Build Specialist Agents** with filtered toolsets.
3. **Construct LangGraph DAG** for the orchestrator.
4. **Integrate** into existing chatbot.
5. **CI updates**.
6. **Security** & **Observability** hooks.

---

7. Advantages of Collection-Centric Agents

- **Separation of concerns** → simpler prompts, smaller context windows per sub-agent.
- **Granular scaling** → heavy gene queries won't starve cancer-lookups.
- **Easier schema evolution** → only the owning agent updates prompts/tests.
- **Targeted fine-tuning** → can train retrieval augmentation per domain.

---

8. Potential Enhancements

- **Add RefinerAgent** to post-process conflicting drug vs gene info.
- **Graph-aware Planner** that leverages schema-linking (PK/FK) to decide join-heavy queries.
- **Vector Cache** of frequent answers per collection; invalidate on collection writes.
